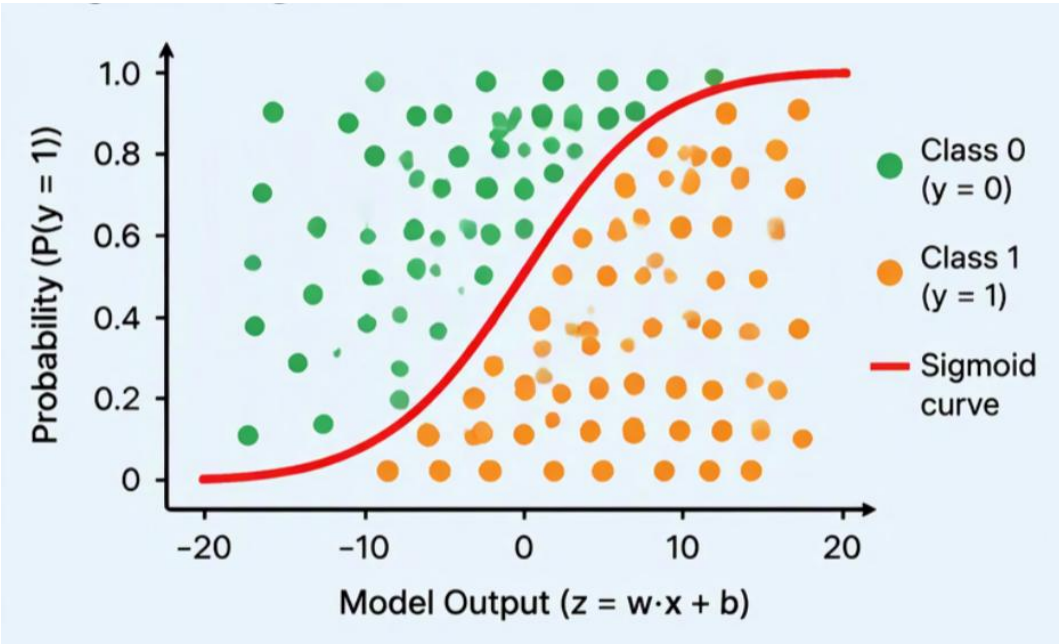
Binary Classification with Logistic Regression



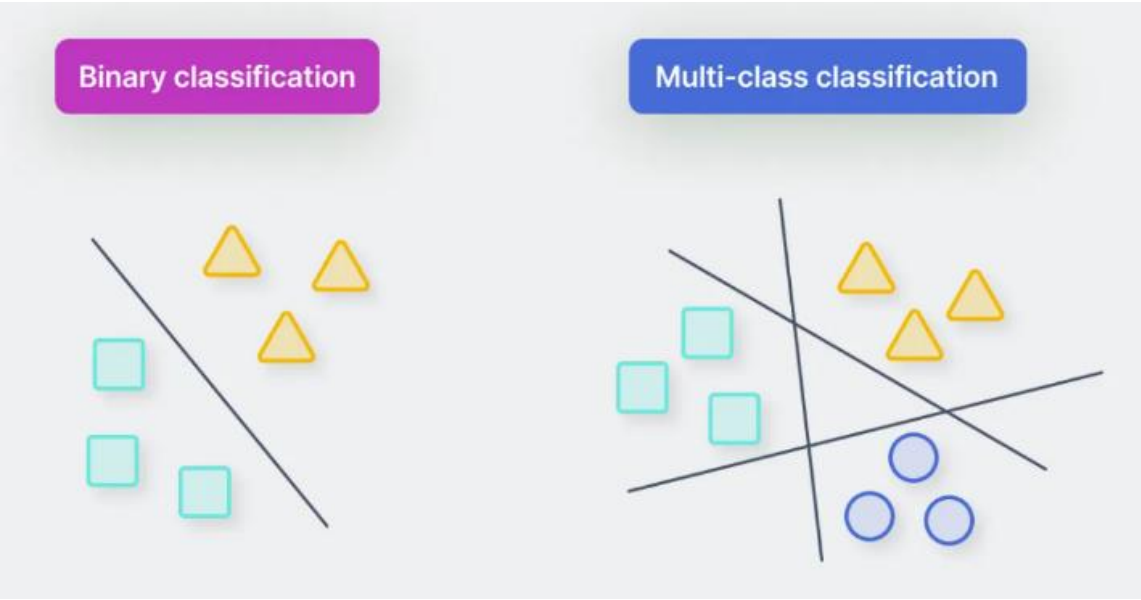
Dr. Abdelkarim Erradi
CSE@QU

With assistance from AI tools, particularly
Microsoft Copilot and Google Gemini

Outline

- Classification
- Logistic Regression
- Classification Model Evaluation
- Handling Class Imbalance
- Model Hyperparameter Tuning

Classification



Classification Models

🎯 **Goal:** predict the **class label** of an input using **supervised learning**

1 Training learn from examples

📘 **Labeled data**
features + known labels

➡

⚙️ **Learning algorithm**
finds patterns

➡

🤖 **Classifier**
the trained model

💡 The algorithm keeps adjusting the model parameters until its predictions match the known labels.

2 Inference predict on new data

📥 **New input**
label unknown ❓

➡

🤖 **Classifier**
the trained model

➡

🏷️ **Predicted label**
most likely class

💡 The trained model assigns the most likely class to data it has never seen.

Classification Examples

Same idea, different labels: **Binary** = 2 classes | **Multi-class** = 3 or more classes



Spam Detection

Is this email spam or not spam? Learns from text and sender data.

Binary



Sentiment Analysis

Is this review positive, negative or neutral?

Multi-class



Medical Diagnosis

Does the patient have the condition or not? Uses symptoms and history.

Binary



Credit Risk Assessment

Is the applicant low, medium or high risk? Uses score, income, debt.

Multi-class



Image Recognition

Is it a cat, dog or bird? Learns visual features from pixels.

Multi-class

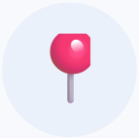
Classification Algorithms



Logistic Regression

Estimates the probability of a class using an S-shaped curve.

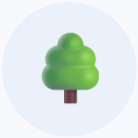
✅ **Best for:** Fast, interpretable baseline



K-Nearest Neighbors

Looks at the K closest examples and takes a vote.

✅ **Best for:** Small datasets, simple intuition



Decision Trees

Asks a series of yes/no questions to reach a decision.

✅ **Best for:** Easy-to-explain rules



Random Forests

Many trees vote together for a more accurate, stable result.

✅ **Best for:** Higher accuracy on tabular data

➕ Many others: SVM, Naive Bayes, Neural Networks 👉 **Next: Logistic Regression**

Logistic Regression

How Logistic Regression Works

1. Input Features



$\mathbf{x} = [x_1, x_2, \ldots, x_n]$

2. Linear Combination

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

3. Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

4. Probability Output

$P(y = 1 \mid \mathbf{x})$

(between 0 and 1)

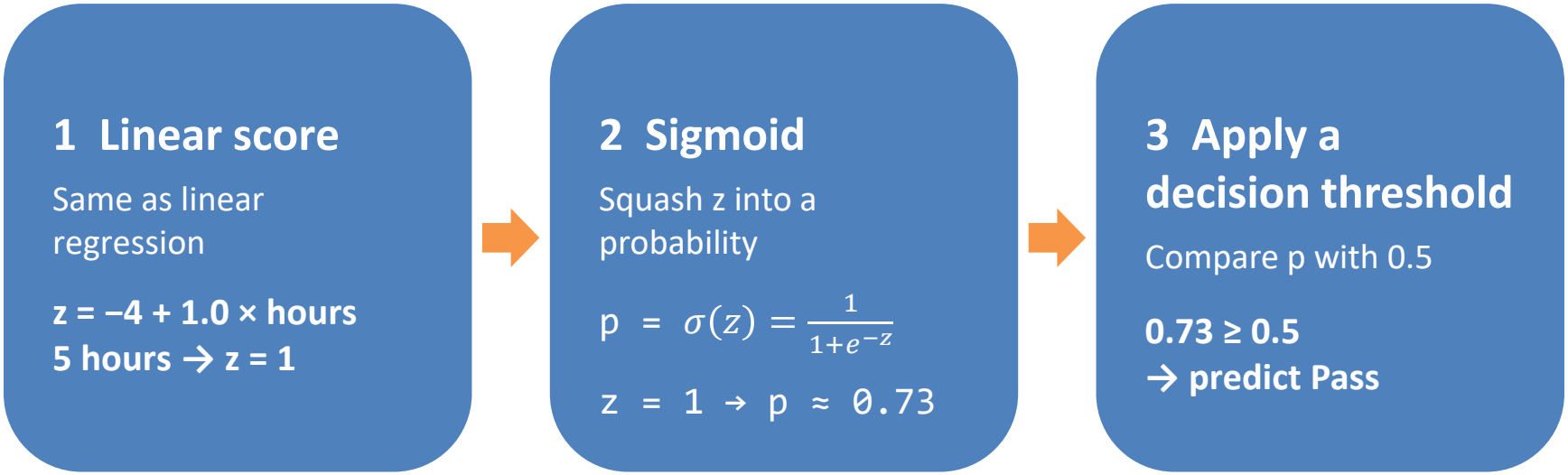
5. Class Prediction

$P \geq 0.5 \rightarrow$ **Class 1**

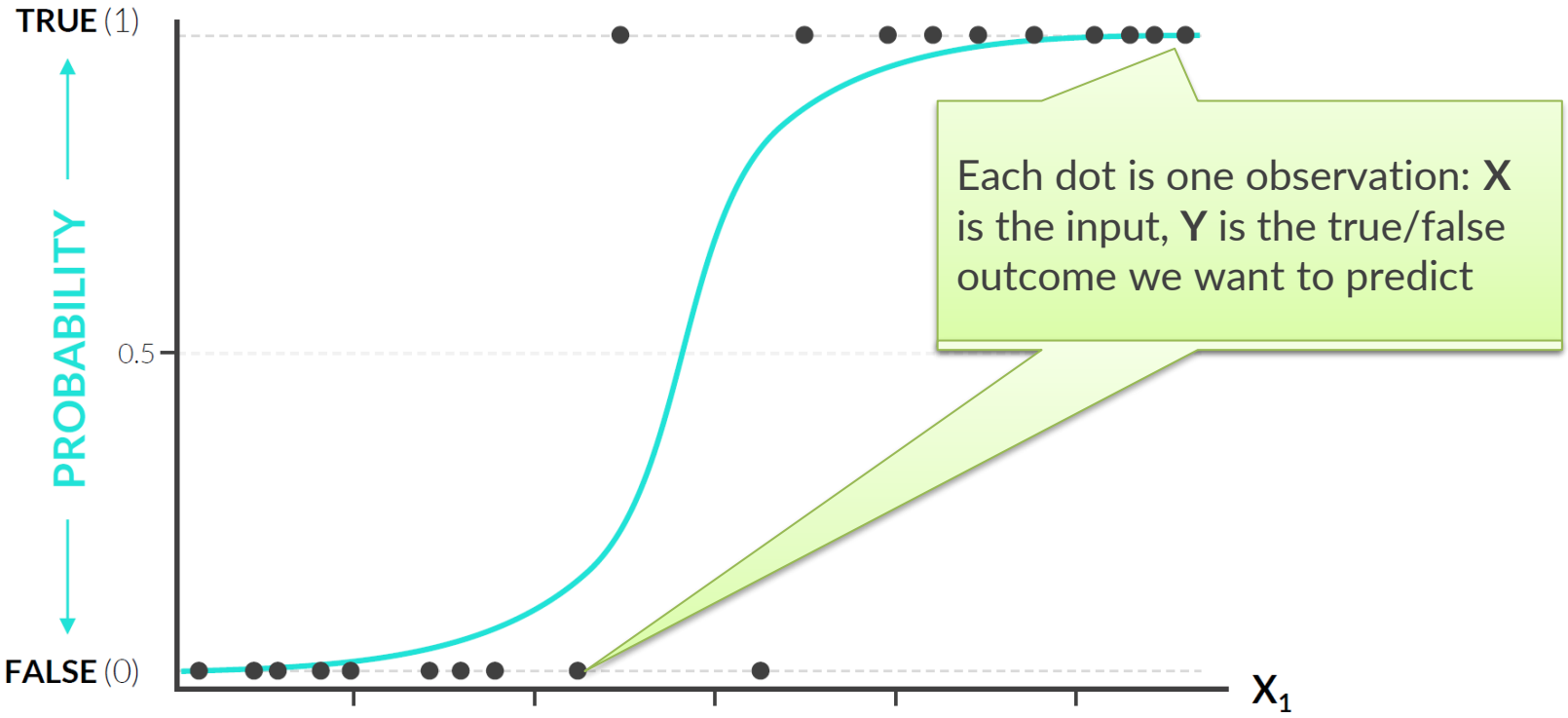
$P < 0.5 \rightarrow$ **Class 0**

From Linear to Logistic Regression

- **Linear regression** predicts a number, e.g. an exam score
- **Logistic regression** starts with a linear score, converts the score to a probability (0 to 1) , then applies a decision threshold.
- **Example:** Will a student pass after studying for 5 hours?

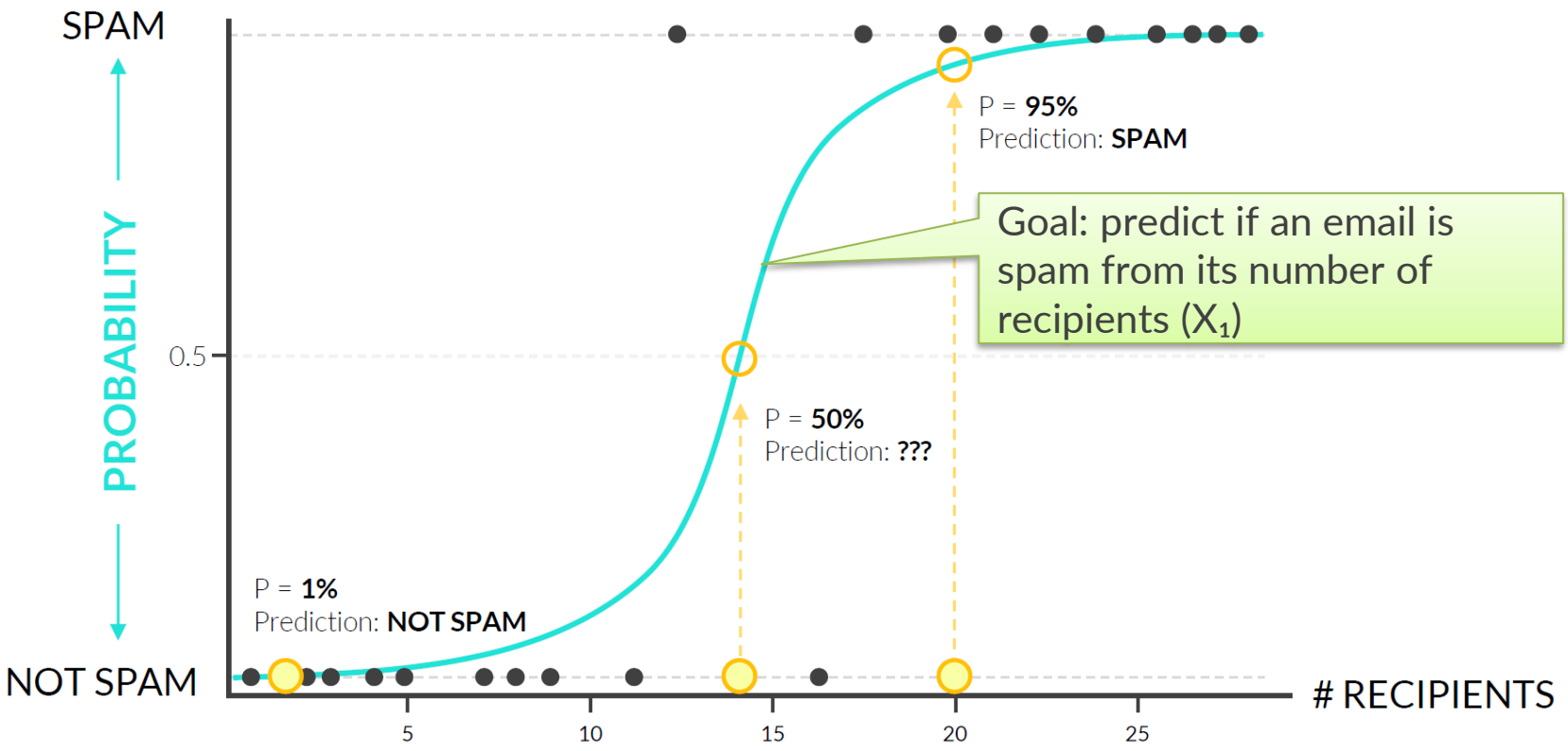


How the S-Curve Works



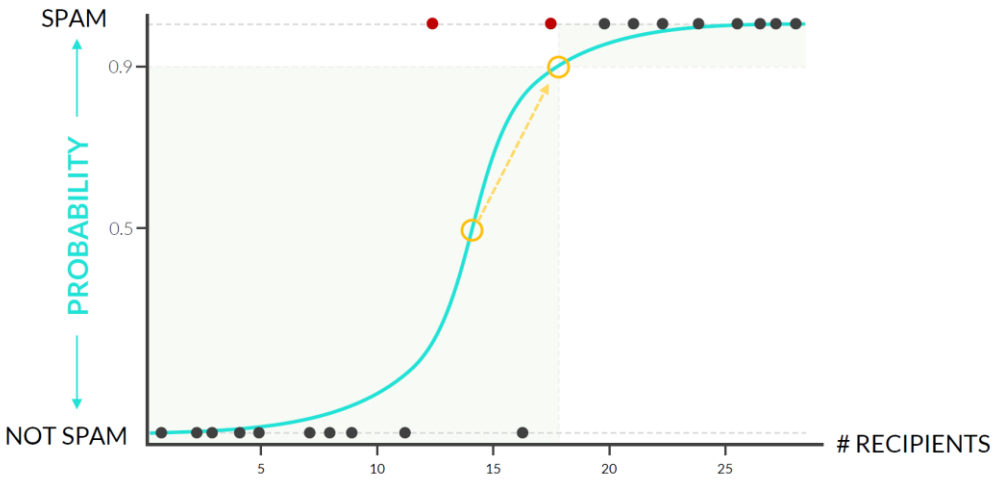
- Logistic regression learns an S-shaped (sigmoid) curve with output values between 0 and 1.
- The curve represents the probability that **Y = TRUE** for a given input X_1
  - Low X_1 → Probability approaches 0 (FALSE).
  - High X_1 → Probability approaches 1 (TRUE).

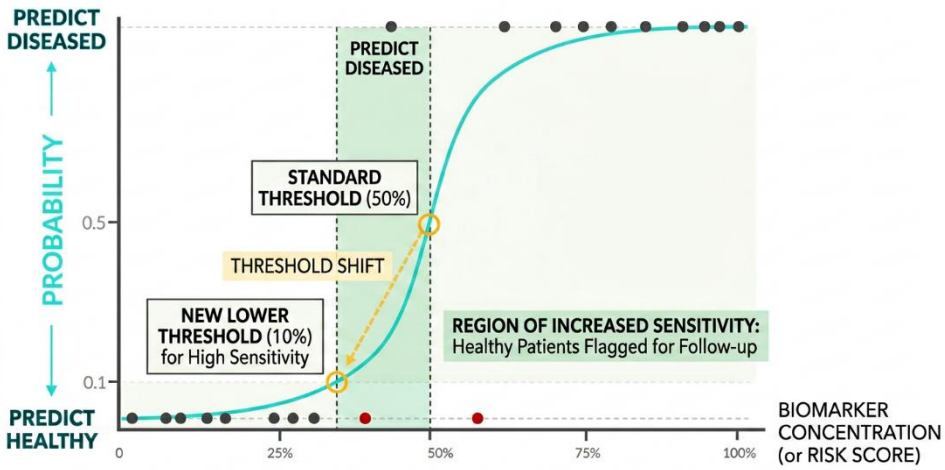
Example: Is This Email Spam?



- Interpret the curve: As the number of recipients (X_1) increases, the predicted probability that the email is spam increases.
- Examples:
  - 2 recipients → P ≈ 1% → Not spam.
  - 20 recipients → P ≈ 95% → Spam.
- Classification rule: Predict Spam if **P > 0.5**; otherwise, predict Not spam. The default classification threshold is typically 0.5.

Is 50% Always the Right Threshold?





- The threshold is a **business decision**: it depends on which mistake costs more

Example 1: Spam filter

Costly mistake: **false positive** *(a false alarm: a real email lands in spam)*

✅ Use a **high threshold** (e.g., 90%): flag as spam only when very sure

Example 2: Cancer screening

Costly mistake: **false negative** = ***a miss***: *a sick patient is told they are healthy*

✅ Use a **low threshold** (e.g., 10–20%): avoid misses by flagging anyone who might be sick. Extra follow-up tests are acceptable

Rule of thumb

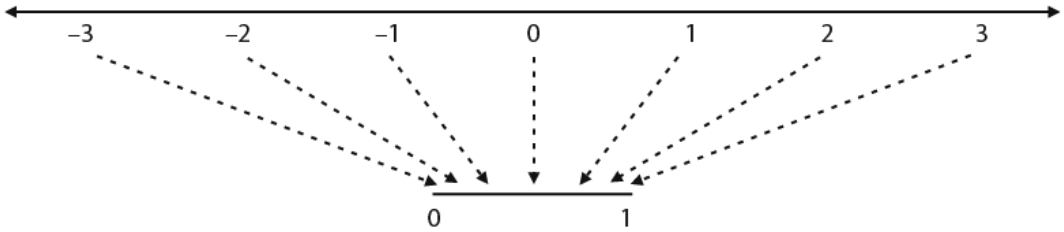
- Costly false positives (false alarms) → raise the threshold.
- Costly false negatives (misses) → lower it.

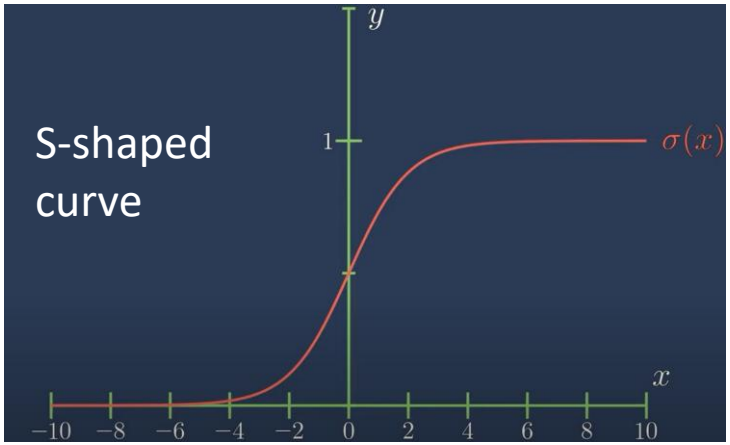
Sigmoid (Logistic) Function

- The **sigmoid (logistic) function** turns any number into a **probability between 0 and 1**
- It is written **σ (sigma)** and defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

  - **e** ≈ 2.718 is a fixed math constant
- Large positive **z**: Probability approaches 1 (Yes).
- Large negative **z**: Probability approaches 0 (No).





Logistic Regression - Model Representation

- Given **m** samples and **n** features, logistic regression predicts the probability of belonging to the positive class (e.g., Will a student pass?)
- The model learns two things:
  - Weights **w** (feature importance)
  - Bias b

X dataset matrix	w parameters vector
$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix}$	$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$

Step 1: Calculate the linear score

$$z = \hat{\mathbf{y}} = X\mathbf{w} + b$$

Where:

$\mathrm{X} \in \mathbb{R}^{\mathrm{m} \times \mathrm{m}}$ input dataset matrix,
$m =$ number of samples,
$n =$ number of input features,
$\mathbf{w} \in \mathbb{R}^{\mathrm{m}}$ (n)−dimensional weight vector,
$b \in \mathbb{R}$ bias term (scalar),
$\hat{\mathbf{y}} \in \mathbb{R}^{\mathrm{m}}$ (m)−dimensional output vector

Step 2: Convert the score into a probability

- Apply the **sigmoid function** to squash the score into a probability between **0 and 1**:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- Predict Yes (positive class) if $\hat{y} >= 0.5$; otherwise, predict No (negative class).

Learning a Logistic Regression Model

- Goal: learn **weights** $\mathbf{w} = [w_1, \ldots, w_m]$ and **bias** b that minimize the **Binary Cross-Entropy** (BCE) cost, which measures the model's prediction error:

$$J = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(\hat{y}^{(i)}\right) + \left(1 - y^{(i)}\right)\log\left(1 - \hat{y}^{(i)}\right)\right]$$

  - m: Number of training examples.
  - $y^{(i)}$: actual label (0 or 1) of example i
  - $\hat{y}^{(i)}$: predicted probability that example i belongs to class 1

Key idea: Training adjusts the weights and bias to minimize the average prediction error across all training examples.

Gradient Descent for Logistic Regression

Objective:

Find the optimal weights $\mathbf{w}$ and bias $\boldsymbol{b}$ that minimize the Binary Cross-Entropy cost.

Algorithm, step by step

1. Initialize weights $\mathbf{w}$ and bias $\boldsymbol{b}$ (e.g., zeros)

2. Predict probabilities with the current values:

$$\hat{y}^{(i)} = \sigma(\hat{\mathbf{y}} = X\mathbf{w} + b\ )$$

3. Compute gradients with respect $\mathbf{w}$ and $\boldsymbol{b}$ to determine how to reduce the cost

4. Update parameters: Update w and b by subtracting their gradients multiplied by the learning rate (α), moving toward a lower cost.

5. Repeat steps 2–4 until the cost stops decreasing (converged ✅)

In practice: scikit-learn runs this loop for you with `model.fit(X, y)`

Example: Will a Student Pass?

- **Goal:** predict whether a student passes (1) or fails (0)
- **Features (X):** hours_studied, attendance_pct
- **Target (y):** passed is a binary outcome (Yes/No), making logistic regression an appropriate choice
- **Data:** 200 students (synthetic, for illustration)

hours_studied	attendance_pct	passed
7.7	89	**1**
4.4	99	**0**
8.6	75	**1**
7.0	57	**1**
0.9	51	**0**

Reading the table

Each row = one student

Inputs (X) = first two columns

Label (y): 1 = pass, 0 = fail

The 5-step workflow



Implementing It with scikit-learn

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

df = pd.read_csv("students.csv")  # 1. load data
X = df[["hours_studied", "attendance_pct"]]
y = df["passed"]

X_train, X_test, y_train, y_test = train_test_split(  # 2. split train / test
    X, y, test_size=0.2, random_state=42, stratify=y)

model = LogisticRegression().fit(X_train, y_train)  # 3. fit the model
model.coef_, model.intercept_  # w = [0.809, 0.031]  b = -6.633
accuracy = model.score(X_test, y_test)  # 4. evaluate: 0.825

# 5. Predict for ONE student: 6 hours of study, 80% attendance
student = pd.DataFrame([[6, 80]], columns=X.columns)
print(model.predict_proba(student))  # [[0.336 0.664]]
# = [P(fail), P(pass)]: 2 classes, so 2 probabilities that add up to 1
```

82.5%

Accuracy: 33 of 40 unseen test students predicted correctly

66%

Chance this student (6 h, 80% attendance) passes. Fail = 34%

≈ 2.2×

Each extra study hour multiplies the odds of passing by e^0.809 ≈ 2.2

≈ 1.03×

Each extra 1% of attendance multiplies the odds by e^0.031 ≈ 1.03 (≈ 1.4× per +10%)

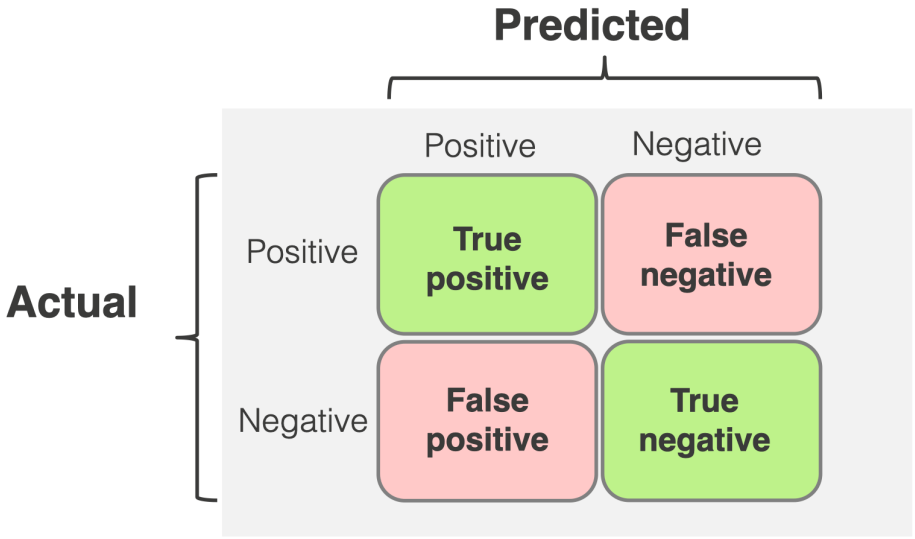
Classification Model Evaluation



Evaluating a Classifier

How good are the predictions?

- Compare predictions with the actual answers
- A confusion matrix counts the right and wrong ones
- Positive = what we look for (e.g., Cat)



✅ **True Positive (TP):** predicted Cat, actually Cat

✅ **True Negative (TN):** predicted Not Cat, actually Not Cat

❌ **False Positive (FP) = false alarm:** predicted Cat, actually Not Cat

❌ **False Negative (FN) = miss:** predicted Not Cat, actually Cat

💡 **Memory aid:** True/False = was the model right? Positive/Negative = what the model predicted.

Worked Example: Is It a Cat? 🐱



Cat	Not Cat	Cat	Not Cat	Cat	Not Cat	Cat	Not Cat	Cat	Cat

Labels above = what the model predicted. 7 of the 10 images are really cats.

	Predicted	
	Cat	**Not Cat**
Actual **Cat**	**TP = 5**	**FN = 2**
Actual **Not Cat**	**FP = 1**	**TN = 2**

7 of 10 predictions are correct. Next: turn these counts into metrics.

How to read the matrix

✅ **TP = 5:** real cats correctly identified as cats

✅ **TN = 2:** dog and bird correctly classified as Not Cat

❌ **FP = 1 (false alarm 🚨):** sheep incorrectly classified as a cat

❌ **FN = 2 (misses ⚠️):** 2 real cats incorrectly classified as Not Cat

4 Key Metrics (Cat Example)

Metric	Question it answers	Formula	Cat example
Accuracy	How often is the model right overall?	`(TP + TN) / All`	(5 + 2) / 10 = 70%
Precision	When it says “Cat”, how often is it right?	`TP / (TP + FP)`	5 / (5 + 1) = 83%
Recall	Of all real cats, how many did it find?	`TP / (TP + FN)`	5 / (5 + 2) = 71%
F1 score	Are precision and recall both good?	`2·P·R / (P + R)`	2·0.83·0.71 / (0.83 + 0.71) ≈ 0.77

$$\text{F1 Score} = \frac{2 \times (\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}$$

Memory aid

Precision: Avoids false alarms (FP).

Recall: Avoids misses (FN).

Watch out: accuracy can mislead

If 95 of 100 images are not cats, a model that always says “Not Cat” is 95% accurate but finds 0 cats.

Which Metric Should I Use?

Precision

Use when false alarms (FP) are costly

Example: spam filter. Don’t block real emails.

Recall

Use when misses (FN) are costly

Example: disease screening. Don’t miss a sick patient.

F1 score

Use when both mistakes matter

Example: fraud detection, or when one class is rare.

Accuracy

Use only if classes are balanced

Both mistakes should cost about the same.

Rule of thumb: costly false alarms → raise the threshold. Costly misses → lower it.

Student Model: Confusion Matrix

```
from sklearn.metrics import confusion_matrix
y_pred = model.predict(X_test)   # 1 = pass, 0 = fail (threshold 0.5)
print(confusion_matrix(y_test, y_pred))  # [[19  3]
                                         #  [ 4 14]]
```

What happened to the 40 test students

		Predicted: Fail	Predicted: Pass
Actual	Fail (22)	**19** True Negative	**3** False Positive
	Pass (18)	**4** False Negative	**14** True Positive

How to read it

Blue cells = correct. 19 + 14 = 33 of 40, so accuracy = 82.5%.

3 false alarms: failing students we predicted would pass.

4 misses: students who passed but we predicted fail.

Student Model: Precision & Recall

```
from sklearn.metrics import precision_score, recall_score, f1_score
precision_score(y_test, y_pred)   # 0.82
recall_score(y_test, y_pred)      # 0.78
f1_score(y_test, y_pred)          # 0.80
```

82%

Precision

`TP / (TP + FP)`
`= 14 / 17`

When we predict Pass, we are right 82% of the time.

78%

Recall

`TP / (TP + FN)`
`= 14 / 18`

We find 78% of the students who really pass.

0.80

F1 score

`2·P·R / (P + R)`

One number that balances precision and recall.

What if we lower the threshold to 0.4 instead of 0.5? Predict Pass when P(pass) ≥ 0.4

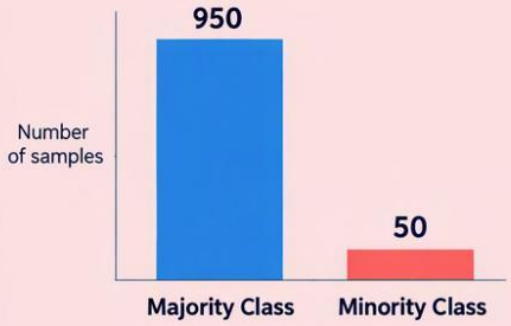
- Recall increases to 83% (15 of 18): We correctly identify more students who will pass.
- Precision drops to 71% (15 of 21): More students are incorrectly predicted to pass.

Key takeaway: Choose the threshold based on the **relative cost of mistakes**: missing a student who needs support (low recall) vs. flagging a student unnecessarily (low precision).

Handling Class Imbalance

1. The Problem

One class has many more samples than the other.

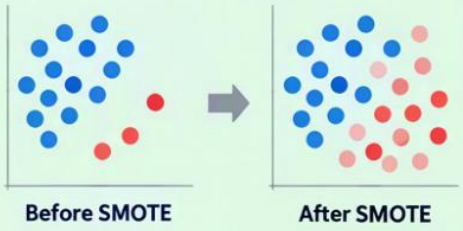


A model trained on this data may be biased toward the majority class and miss important minority cases.

2. Two Effective Solutions

1 Oversampling with SMOTE

Create synthetic samples for the minority class (using SMOTE) to balance the data.



- Majority class (original)
- Minority class (original)
- Minority class (synthetic - SMOTE)

2 Using class_weight='balanced'

Give higher weight to the minority class so that the model pays more attention to it during training.

```
from sklearn.linear_model
import LogisticRegression

model = LogisticRegression(
    class_weight='balanced'
)
```

The algorithm automatically assigns higher weights to the minority class based on the class frequencies.

Class Imbalance: The Accuracy Trap

- **Example:** predicting which students will fail. Of 1,000 students, 950 pass and only 50 fail.
- **A lazy model** predicts “pass” for everyone.
- **Its accuracy is 95%** (950 of 1,000 correct). That looks great!
- **But it finds 0 of the 50 failing students** (Recall = 0%), the very students we want to help.
- **Why?** Training minimizes total mistakes, so the majority class dominates.

Remember: check Recall and F1 for the minority class, not just accuracy.

Fix 1: class_weight='balanced'

```python
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

# Baseline: every mistake counts the same
base = LogisticRegression(max_iter=1000)
base.fit(X_train, y_train)

# Balanced: minority-class mistakes count more
bal = LogisticRegression(class_weight='balanced', max_iter=1000)
bal.fit(X_train, y_train)

# Compare both on the SAME test set
print(classification_report(y_test, base.predict(X_test)))
print(classification_report(y_test, bal.predict(X_test)))

# weight = n_samples / (n_classes x class_count)
```

0.56 | 5.0
Weights for 900 majority vs 100 minority samples

No new data
Same rows. Only the importance of each class changes

≈ 9× cost
A missed minority case is penalized about 9× more

Not magic
Pays more attention to the minority class, but overall performance may not improve

Fix 2: Oversampling with SMOTE

Oversampling

Give the minority class more samples

Before: 900 vs 100. After: 900 vs 900 (+800 minority samples).

RandomOverSampler

Duplicates existing minority rows

Simple, but the model may memorize the repeats (overfitting).

SMOTE

Creates synthetic minority samples

Interpolates between a minority point and its minority-class neighbors.

Extra library needed

imbalanced-learn (imblearn)

Install with: pip install imbalanced-learn

Golden rule: apply SMOTE to the training data only, never to the test set.

SMOTE in Code

```
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

# SMOTE sits INSIDE the pipeline: it only touches training data
model = make_pipeline(
    SMOTE(random_state=42),
    StandardScaler(),
    LogisticRegression(max_iter=1000))

model.fit(X_train, y_train)     # resamples X_train only
y_pred = model.predict(X_test)  # test set stays untouched

# Cross-validation: SMOTE runs separately inside each fold
cross_val_score(model, X_train, y_train, scoring="f1", cv=5)

# By hand: X_bal, y_bal = SMOTE().fit_resample(X_train, y_train)
```

100 → 900

Minority samples after SMOTE (+800 synthetic)

Train only

Never resample the test set

Use a Pipeline

In cross-validation, resampling happens per training fold

class_weight vs. Oversampling

Aspect	class_weight='balanced'	Oversampling (SMOTE)
What changes?	Importance of each class	Number of training samples
Adds samples?	No	Yes
Duplicates or synthetic?	Neither	RandomOverSampler duplicates; SMOTE creates synthetic
Extra library?	No	Yes: imbalanced-learn
Main risk	May increase false positives	Synthetic noise or overfitting

Key distinction

class_weight: make minority-class errors more costly during training.

SMOTE: give the model additional synthetic minority examples.

Which Should You Use?

Step 1: Baseline

Logistic Regression, no balancing

Your reference point. Always run it first.

Step 2: class_weight

class_weight='balanced'

Teach and try this first: simple, no extra library.

Step 3: SMOTE

Oversample the minority class

Try it when imbalance is large and you have enough representative minority samples.

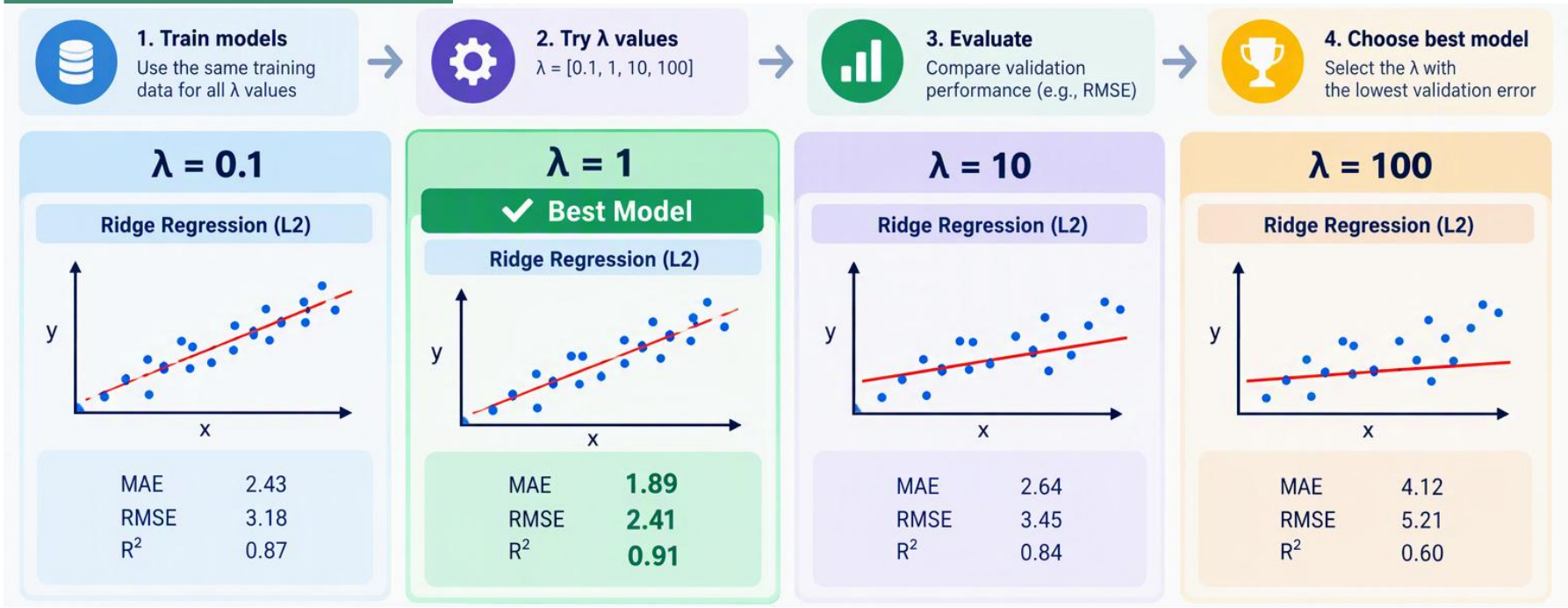
Compare fairly

Same test set, same metrics

Confusion matrix, Precision, Recall, F1. Cross-validate; weigh false positives vs false negatives.

Key takeaway: class_weight='balanced' makes Logistic Regression pay more attention to the minority class, reducing bias toward the majority. It does not guarantee better overall performance.

Model Hyperparameter Tuning



Hyperparameters and C

- **Hyperparameter:** a setting we choose before training. The model does not learn it.
- **C controls regularization strength:** it is an inverse, so smaller C = stronger penalty.
- **Small C (e.g. 0.01):** simpler model, may underfit.
- **Large C (e.g. 100):** fits the training data closely, may overfit.
- **Default:** L2 regularization with C = 1.0. Tune C first.

Remember: choose C with cross-validation, never with the test set.

What We Will Search

C

[0.01, 0.1, 1, 10, 100]

Controls L2 regularization strength.

class_weight

[None, 'balanced']

Handles imbalanced classes.

Grid size

5 × 2 = 10 combinations

GridSearchCV tries every combination.

Cross-validation

cv=5 → 50 model fits

Each hyperparameter combination is evaluated using 5 training folds, producing 5 scores.

L2 (the default) stays fixed. We tune only these two settings.

GridSearchCV in Code

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# 1. Scale features, then Logistic Regression (L2 by default)
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))

# 2. Values to try (step name + __ + parameter name)
param_grid = {
    "logisticregression__C": [0.01, 0.1, 1, 10, 100],
    "logisticregression__class_weight": [None, "balanced"]}

# 3. Try all 10 combinations with 5-fold cross-validation
grid = GridSearchCV(pipe, param_grid, cv=5, scoring="f1")
grid.fit(X_train, y_train)   # training data only

# 4. Inspect the winner, then test it once
print(grid.best_params_, grid.best_score_)
print(grid.score(X_test, y_test))   # F1 on unseen data
```

10 × 5 = 50

Combinations × folds = model fits

Train only

The search never sees the test set

scoring='f1'

Pick a metric that fits your goal

Reading the Results

best_params_

The winning settings

Example: C = 1, class_weight = 'balanced' (illustrative).

best_score_

Average CV score of the winner

Compare settings with it. It is not the final result.

Final check

Test the best model on the test set

Use the confusion matrix, Precision, Recall and F1.

Stay practical

A bigger grid takes longer

Start with a few sensible values.

Key takeaway: tune C and class_weight with cross-validation, then test the best model on the test set.

Summary

Logistic Regression	Predicts the probability of a class using the sigmoid; a threshold (default 0.5) turns it into a label.
Training	Learns weights by minimizing log-loss with gradient descent; scikit-learn: fit, predict, predict_proba.
Evaluation	Use the confusion matrix, Precision, Recall and F1, not accuracy alone.
Class imbalance	Accuracy can mislead. Fix with class_weight='balanced' or SMOTE (training data only).
Tuning	Use GridSearchCV with cross-validation to tune C and class_weight, then evaluate the best model on the test set.
Golden rule	Never tune or resample using the test set.

Resources

- **User guide:** Logistic regression
  https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- **Evaluation:** Confusion matrix, Precision, Recall, F1
  https://scikit-learn.org/stable/modules/model_evaluation.html
- **Tuning:** GridSearchCV and cross-validation
  https://scikit-learn.org/stable/modules/grid_search.html
- **Imbalanced data:** imbalanced-learn (SMOTE)
  https://imbalanced-learn.org/
- **Course:** ML Specialization (Andrew Ng), classification module
  https://www.deeplearning.ai/specializations/machine-learning